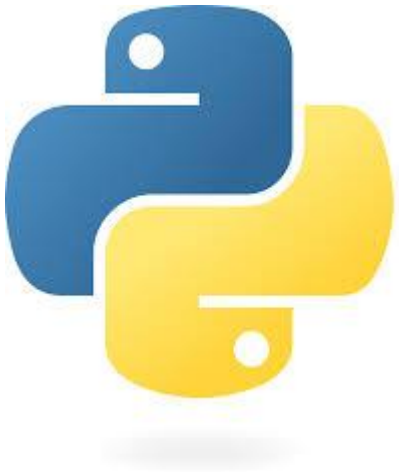
Python



PROGRAMMING FOR EVERYONE

Eng. Omar Shaqra

Seaborn Basics & Styling

Seaborn is a **Python data visualization library** built on top of **Matplotlib**. It makes it much easier to create **statistical graphics** and visualize data in a more **attractive and informative way**. Think of it as Matplotlib, but prettier and more high-level.

Key Features

- **Beautiful default styles** – you get a clean, attractive look without much effort.
- **High-level interface** – simple functions for complex plots.
- **Works well with Pandas** – can accept DataFrames directly.
- **Built-in themes and color palettes** – great for aesthetics.
- **Supports statistical plotting** – regression, boxplots, violin plots, etc.

Seaborn Basics & Styling

Built on Top of Matplotlib

- Seaborn is **a high-level Python visualization library** for statistical graphics.
- Internally, it **uses Matplotlib to create plots**.
- Every Seaborn plot is essentially a **Matplotlib Figure and Axes object**.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Scatter plot: Study Hours vs Grade, colored by Passed
sns.scatterplot(data=df,
                x='Study_Hours',
                y='Grade',
                hue='Passed',
                size='Age',
                palette='deep',
                sizes=(50, 200))

plt.title("Grade vs Study Hours (Size = Age, Color = Passed)")
plt.xlabel("Study Hours per Week")
plt.ylabel("Grade")
plt.grid(True, alpha=0.5)
plt.show()
```

Styles, Contexts & Color Palettes

Seaborn lets you set global style once → all plots look consistent and beautiful No need to customize every single plot manually

Setting Styles

```
# Most common & clean style
sns.set_style("whitegrid")      # adds light grid, white background

# Other popular options
# sns.set_style("darkgrid")     # dark background with grid
# sns.set_style("white")        # clean, no grid
# sns.set_style("dark")         # dark background, no grid
# sns.set_style("ticks")        # small ticks on axes
```

Styles, Contexts & Color Palettes

Setting Context (font size & scaling)

```
# Best for presentations / notebooks
sns.set_context("talk", font_scale=1.3)

# Options:
# "paper"    → smallest (default for publications)
# "notebook" → good for Jupyter
# "talk"     → larger fonts for presentations
# "poster"   → largest
```

Color Palettes

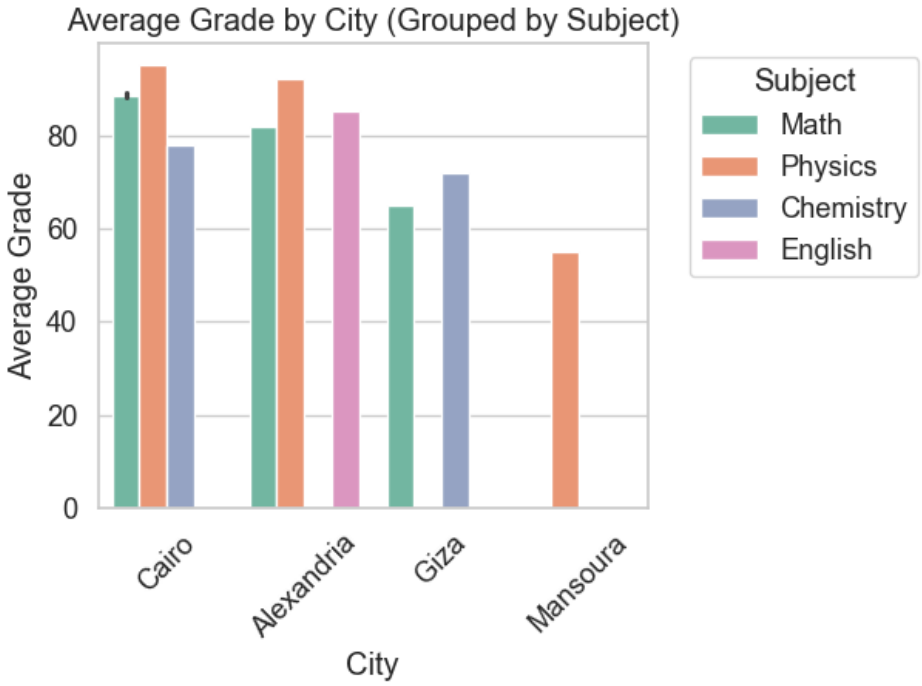
```
# Set once for all plots
sns.set_palette("husl")          # bright, distinct colors

# Popular palettes you'll use a lot
# "deep"      → strong, professional
# "muted"     → soft, elegant
# "Set2"      → nice for categories
# "viridis"   → sequential gradient (great for heatmaps)
# "coolwarm"  → diverging (good for correlations)
```

Styles, Contexts & Color Palettes

Feature / Aspect	scatterplot	lineplot	relplot
Main Purpose	Show individual points / relationships	Show trends / continuous change over time or ordered variable	Most flexible: scatter or line + faceting (subplots)
Plot Type	Scatter (dots)	Connected lines (with optional points)	Wrapper — can be scatter or line + rows/cols facets
Best For	Correlations, clusters, distributions	Time series, trends, averages with CI	Exploring relationships across categories (e.g. by group, facet)
Supports Hue (color)	Yes	Yes	Yes
Supports Size	Yes	Yes (but less common)	Yes
Supports Style (markers)	Yes	Yes (marker on points)	Yes
Confidence Intervals	No	Yes (default)	Yes (if kind="line")
Faceting (col/row)	No	No	Yes (the killer feature)
Figure-Level or Axes-Level	Axes-level	Axes-level	Figure-level (creates own figure)
When to use	Simple scatter with categories	Trends over ordered axis (time, age…)	Complex analysis with subplots by category
Example with students data	Study_Hours vs Grade, hue=Passed	Grade vs Age (sorted), hue=City	Study_Hours vs Grade, col=Subject, hue=Passed

Styles, Contexts & Color Palettes

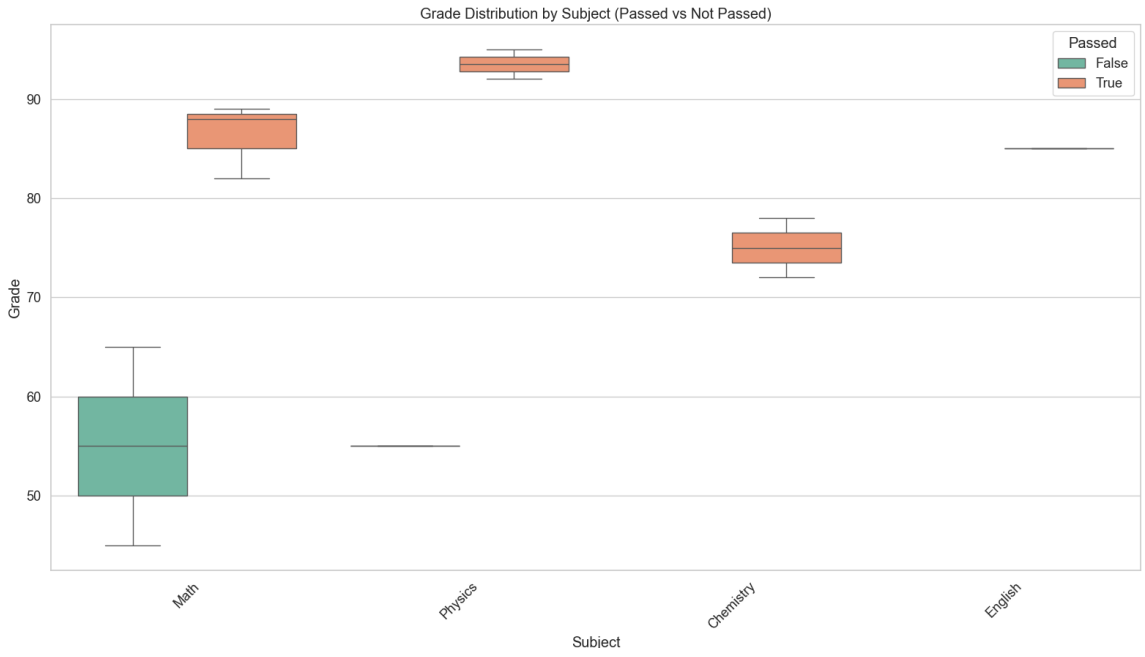
sns.barplot




```python
# Apply style from previous slides (for consistency)
sns.set_style("whitegrid")
sns.set_context("notebook", font_scale=1.2)
sns.set_palette("Set2")

# Barplot: Average Grade by City, colored by Subject
sns.barplot(data=df,
            x="City",
            y="Grade",
            hue="Subject",
            errorbar="ci")  # confidence interval

plt.title("Average Grade by City (Grouped by Subject)")
plt.xlabel("City")
plt.ylabel("Average Grade")
plt.xticks(rotation=45)
plt.legend(title="Subject", bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```

Styles, Contexts & Color Palettes

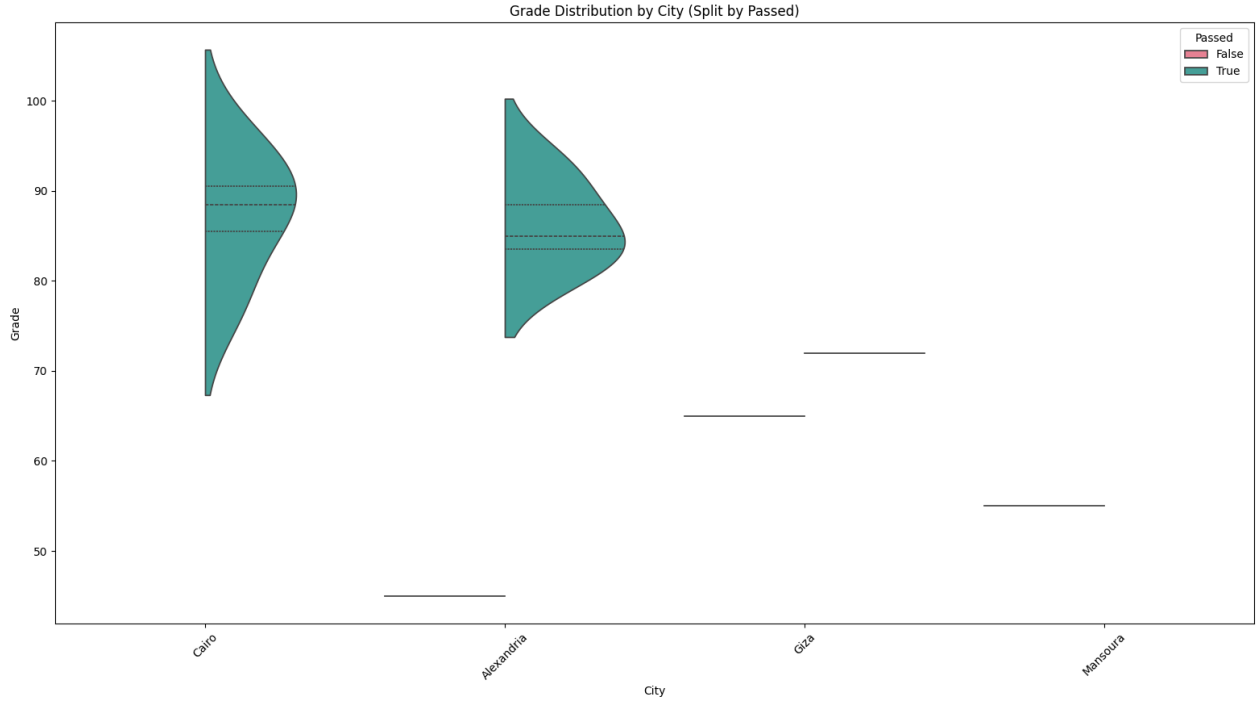
sns.boxplot




```python
# Apply style from previous slides (for consistency)
sns.set_style("whitegrid")
sns.set_context("notebook", font_scale=1.2)
sns.set_palette("Set2")

# Boxplot: Grade by Subject, colored by Passed
sns.boxplot(data=df,
            x="Subject",
            y="Grade",
            hue="Passed",
            palette="Set2")

plt.title("Grade Distribution by Subject (Passed vs Not Passed)")
plt.xlabel("Subject")
plt.ylabel("Grade")
plt.xticks(rotation=45)
plt.show()
```

Styles, Contexts & Color Palettes

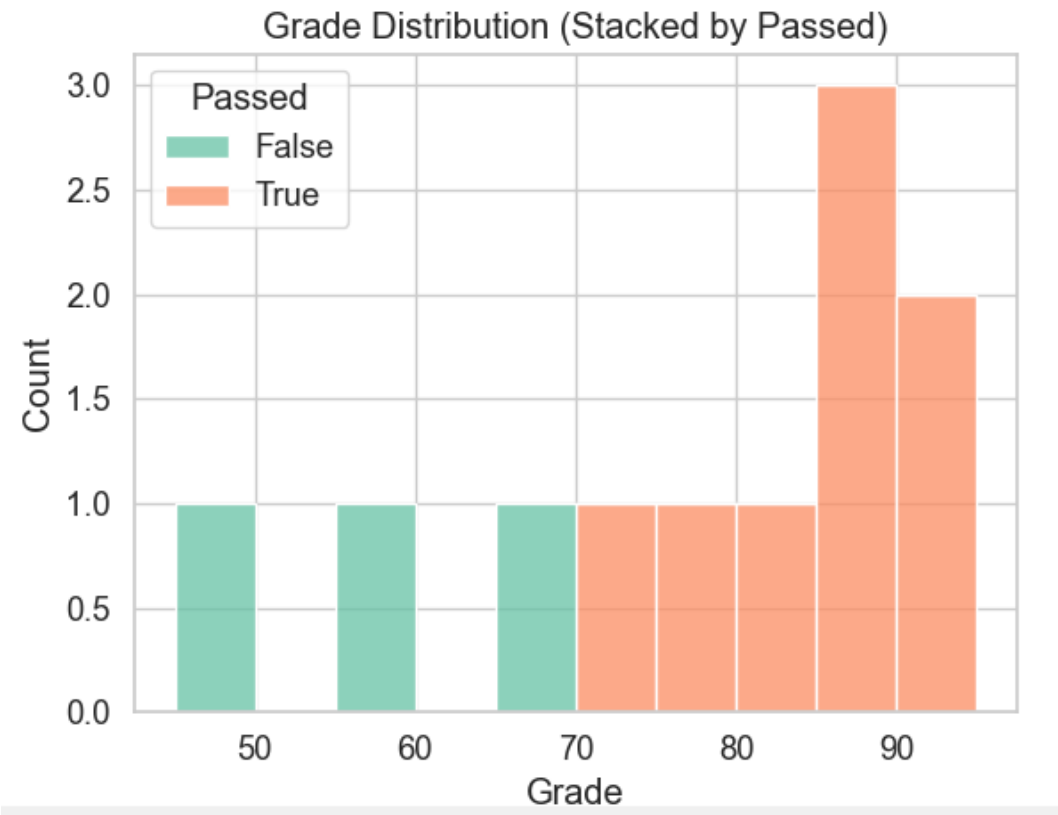
sns.violinplot




```python
# Violin plot: Grade by City, split by Passed
sns.violinplot(data=df,
               x="City",
               y="Grade",
               hue="Passed",
               split=True,
               inner="quartile",
               palette="husl")

plt.title("Grade Distribution by City (Split by Passed)")
plt.xlabel("City")
plt.ylabel("Grade")
plt.xticks(rotation=45)
plt.show()
```

Styles, Contexts & Color Palettes

sns.histplot




```python
# Apply style from previous slides (for consistency)
sns.set_style("whitegrid")
sns.set_context("notebook", font_scale=1.2)
sns.set_palette("Set2")

# Histogram: Distribution of Grades
sns.histplot(data=df,
             x="Grade",
             hue="Passed",
             multiple="stack",
             bins=10)

plt.title("Grade Distribution (Stacked by Passed)")
plt.xlabel("Grade")
plt.ylabel("Count")
plt.show()
```

Proj

1. Load the data from the API Use requests or pd.read_json to fetch data from this endpoint:
   - https://dummyjson.com/users?limit=100

2. Basic Data Exploration

   •Shape of the DataFrame (rows, columns)
   •List of all column names
   •Data types of each column (df.dtypes)
   •Number of missing values per column (isnull().sum())
   •Number of duplicate rows (duplicated().sum())
   •Summary statistics for numeric columns (describe())
   •Value counts for important categorical columns (e.g. gender, bloodGroup, eyeColor, role, address.country)

3. Data Cleaning / Preparation (if needed)

   •If any important column is missing (e.g. country), extract it from nested columns like address using .apply() and json.loads if necessary
   •Handle missing values in age, height, weight (fill with mean/median or drop if few)

Proj

Analysis

1. What is the average age of users?
2. Average age by gender?
3. Number of users per gender?
4. Top 10 cities with the most users?
5. Average height and weight overall?
6. Is there any obvious relationship between age and height/weight?

Prove your answer through Seaborn Visualizations (create at least 5 plots)